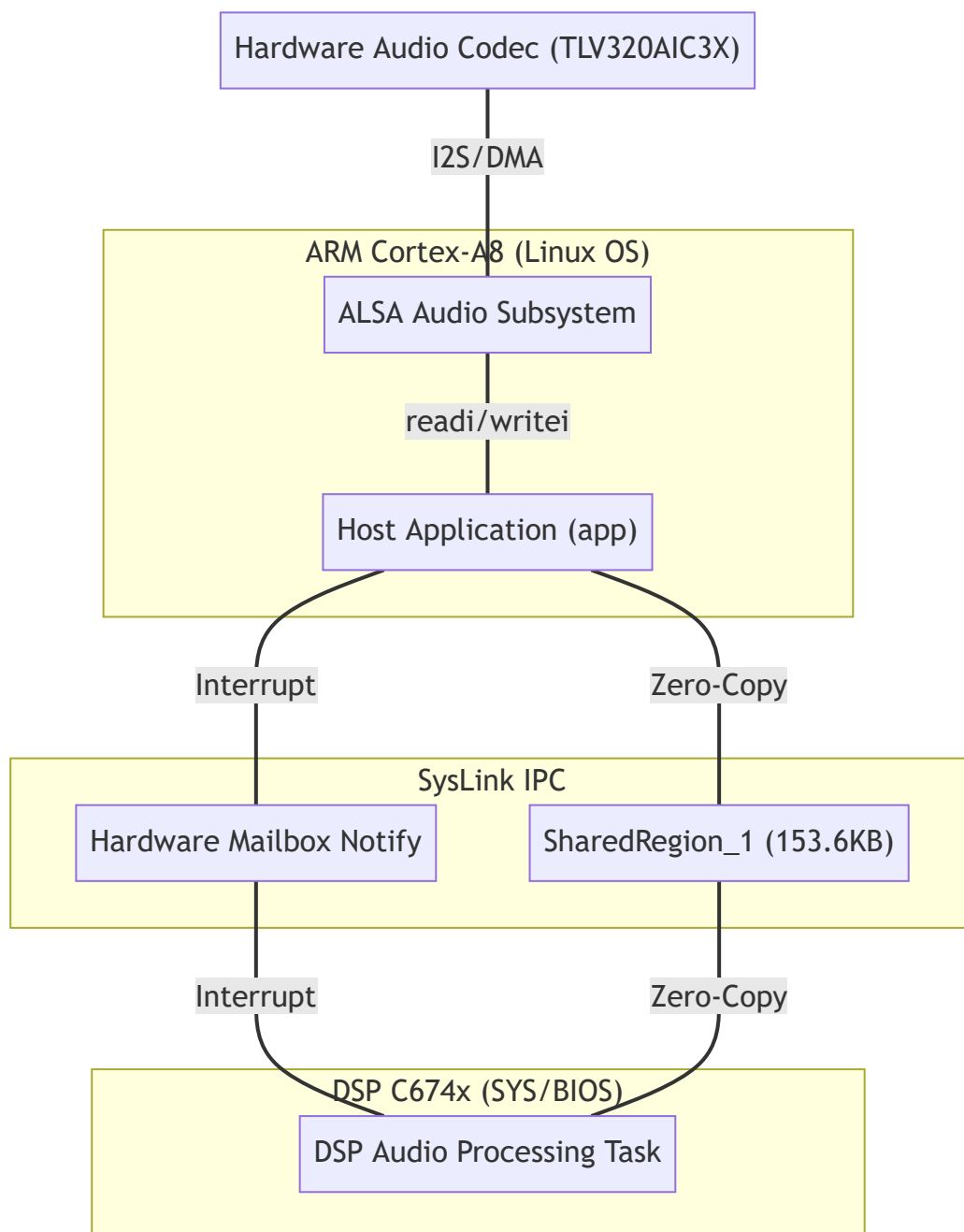


DM8168 异构多核音频流控系统详细设计文档 (System Design)

本文档旨在全面且深入地阐述 DM8168 (ARM Cortex-A8 + DSP C674x) 双核音频处理底座的核心架构、内存管理、IPC 信令机制以及高级抗撕裂流控策略。文档中预留了专门的图表占位符，以便在后续架构演进中插入详细的时序图、状态机图与内存分布图。

1. 系统架构总览 (Architecture Overview)

本系统采用非对称多处理 (AMP, Asymmetric Multiprocessing) 架构，通过异构核心的物理级解耦，实现“调度管理”与“实时计算”的完美分离。

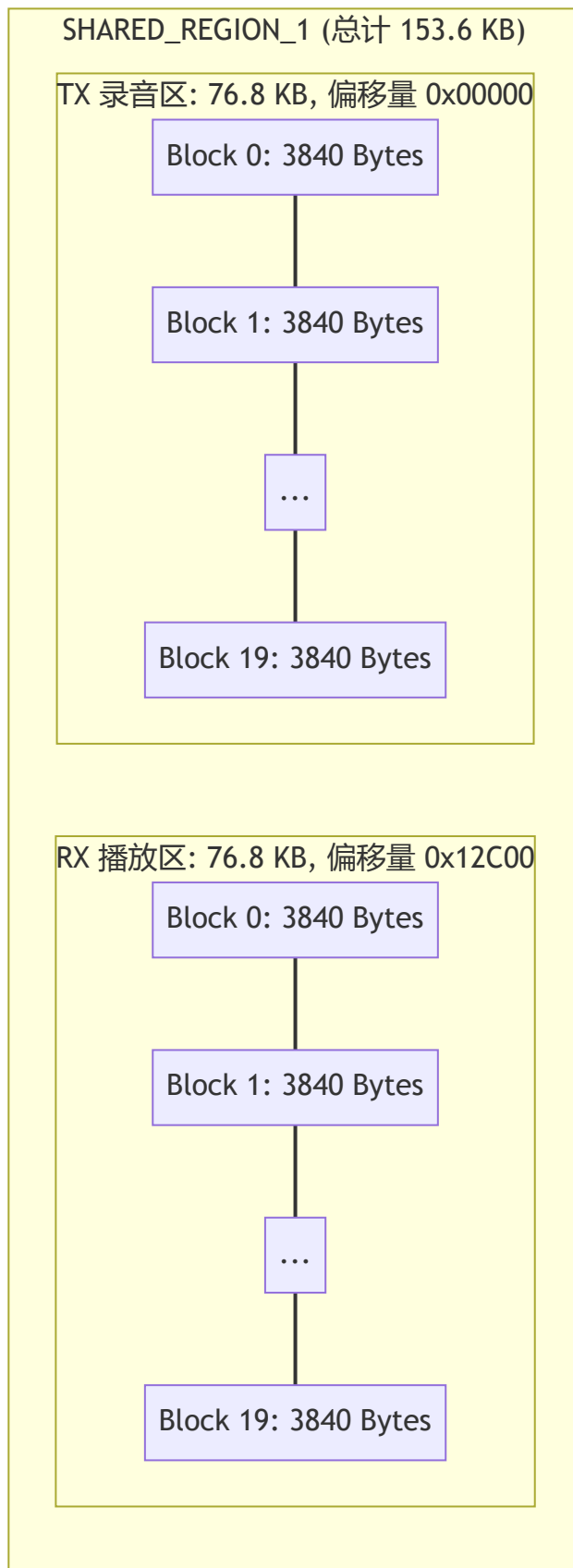


- **Host 端 (ARM Cortex-A8)**
 - **职责:** 运行标准 Linux 操作系统。全权接管 ALSA 音频子系统的复杂驱动交互、网络协议栈、文件 I/O 以及外部硬件中断捕获。
 - **特性:** 由于非实时 OS 存在调度延迟, Host 端被设计为“容错型”数据搬运工, 通过大容量缓冲池来抵消 Linux 调度带来的抖动。
- **Server 端 (DSP C674x)**
 - **职责:** 运行 TI SYS/BIOS 实时操作系统 (RTOS)。处于“无外设、无抢占、纯数学”的理想真空环境中, 专注于执行高算力的数字信号处理算法 (如 AEC 降噪、AGC 自动增益、1024阶 FIR 滤波等)。
- **双核桥梁 (SysLink)**

- **物理层**: `SharedRegion` (基于高速内部 RAM 或专享 DDR 划分的非 Cache 一致性内存)。
 - **信令层**: `Notify` (基于底层硬件 Mailbox 邮箱寄存器的超低延迟硬中断)。
-

2. 物理内存管理与环形缓冲设计 (Memory Layout & Ring Buffer)

为了消除传统双核通信中的 `memcpy` 性能损耗，本系统采用**零拷贝** (Zero-Copy) 物理共享内存池设计。



2.1 音频规格与缓存深度计算

- 采样规格: 48,000 Hz, Signed 16-bit, Stereo (4 Bytes/Frame)。

- **ALSA 中断周期:** `PERIOD_FRAMES = 960` , 即底层硬件每 20ms 触发一次数据搬运。
- **单块大小 (Block Size):** `960 Frames * 4 Bytes = 3840 Bytes` 。
- **抗抖动深度 (Block Count):** 单向设计为 `20` 个 Block, 总计 **400 毫秒** 的物理级缓冲池深度。即使 Linux 内核锁死高达 300 毫秒, 底层 DMA 依旧有足够的数据连续发声。

2.2 内存空间划分

系统总申请物理内存大小为 153.6 KB (`FULL_BUFFER_SIZE`), 均分两区:

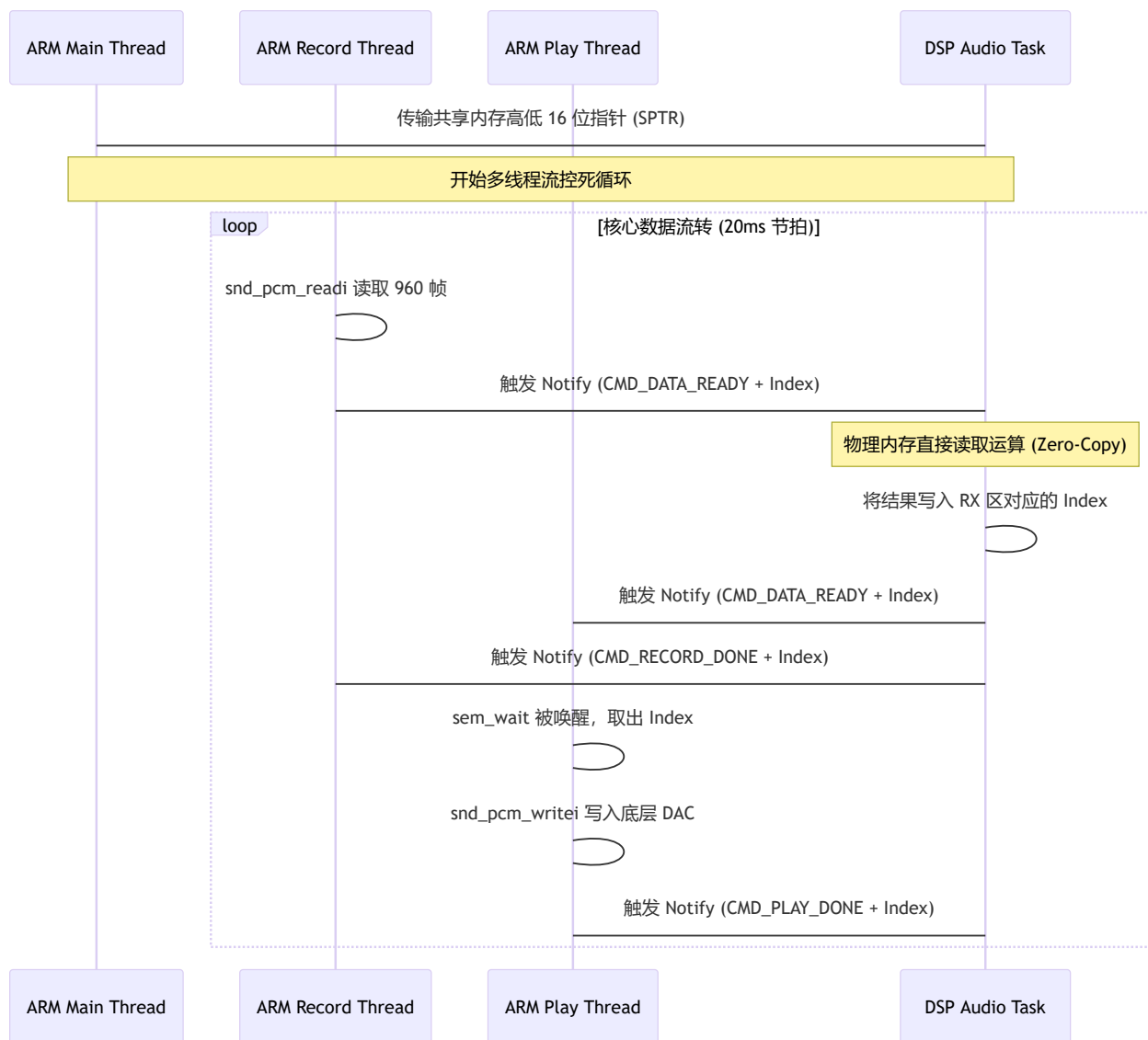
1. **TX 录音区** (`0x000000` - `0x12BFF`): 共 76,800 Bytes。存放 ARM 从外设采集的原始 PCM 数据。
2. **RX 播放区** (`0x12C00` - `0x257FF`): 共 76,800 Bytes。存放 DSP 算法处理完毕、准备送给 DAC 的音频数据。

2.3 物理-虚拟地址转换屏障

Host (Linux) 操作的是带 MMU 映射的虚拟地址, 而 DSP 操作的是物理地址。所有跨核传递的数据指针必须通过 `SharedRegion_getSRPtr()` 转换为与地址空间无关的 32 位 `SRPtr` 标识, 对方接收后再通过 `SharedRegion_getPtr()` 翻译为本地可用地址。

3. 多线程模型与无锁调度 (Threading Model & Lock-free IPC)

Host 端设计了三个高并发线程, 依靠互斥信号量 (`sem_t`) 与 IPC 信令实现无锁 (Lock-free) 的环形队列调度。



3.1 Host 端三线程模型

1. **Main 线程 (控制面)**: 负责初始化 ALSA 参数、申请共享内存、向 DSP 发送初始化地址, 以及捕获退出信号执行清理。
2. **Record 线程 (生产端)**: 调用 `snd_pcm_readi` 阻塞读取音频, 读取完成后通过 `CMD_APP_TO_SERVER_DATA_READY` 唤醒 DSP。受 `empty_in` 信号量约束 (初始值为 `BLOCK_COUNT`)。
3. **Play 线程 (消费端)**: 受 `full_out` 信号量约束 (初始值为 0), 被 DSP 唤醒后, 调用 `snd_pcm_writeti` 写入底层驱动, 完毕后发送 `CMD_APP_TO_SERVER_PLAY_DONE` 归还所有权。

3.2 IPC 信令编码协议

TI Notify Payload 只有 32-bit, 为最大化信息量, 系统将其拆分为:

- **高 16 位 (Command)**: 状态机指令。

- **低 16 位 (Index)**: 操作的目标内存块索引 (0 ~ 19)。

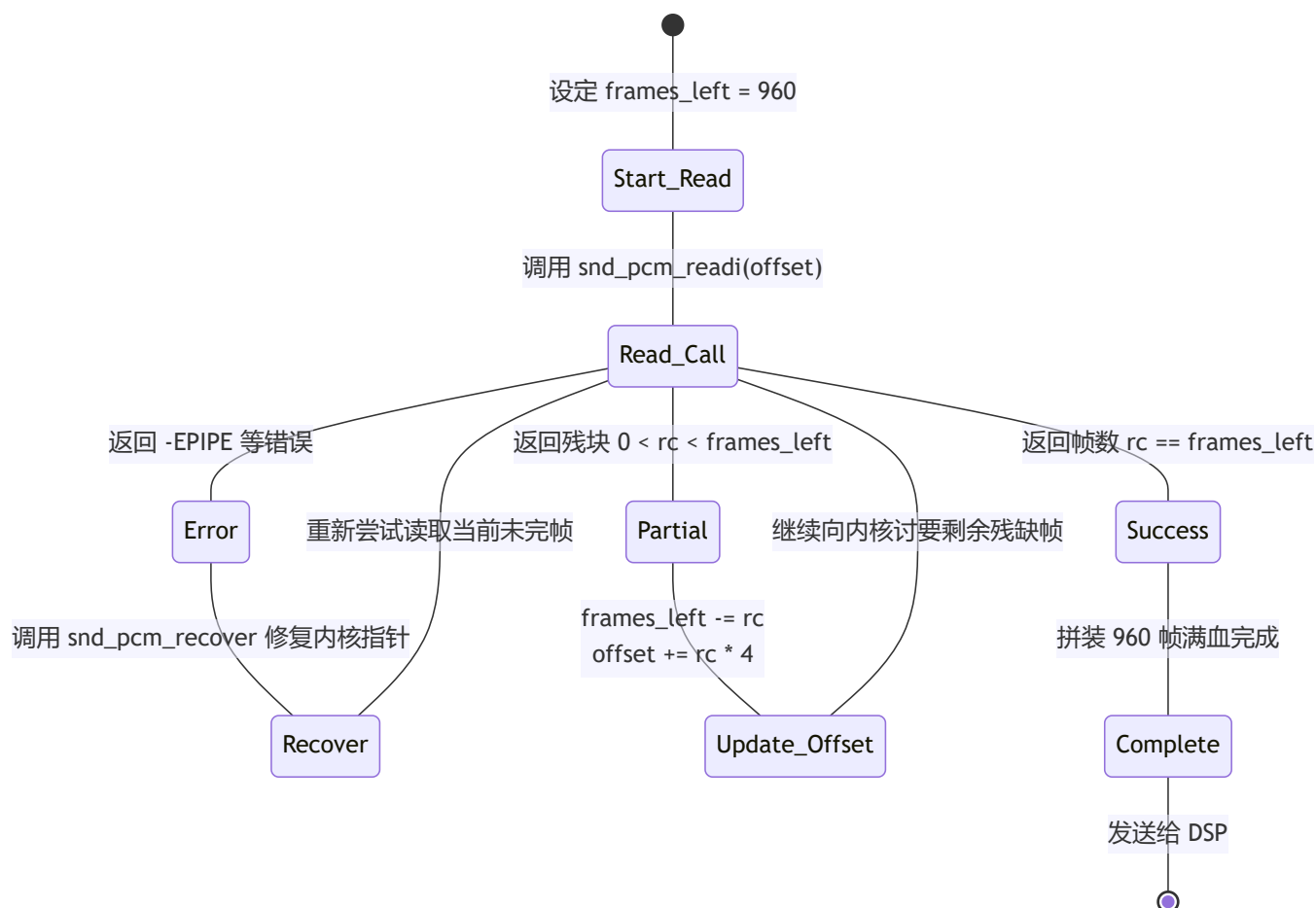
核心数据驱动指令 (4 次轮转) :

```
#define CMD_APP_TO_SERVER_DATA_READY 0x0001 // [0x0001][Index]: ARM 采集完毕, 交由 DSP  
运算  
#define CMD_APP_TO_SERVER_PLAY_DONE 0x0002 // [0x0002][Index]: ARM 播放完毕, 该块可被  
覆写  
#define CMD_SERVER_TO_APP_DATA_READY 0x0003 // [0x0003][Index]: DSP 处理完毕, 通知 ARM  
播放  
#define CMD_SERVER_TO_APP_RECORD_DONE 0x0004 // [0x0004][Index]: DSP 消费完录音块, 归还  
给 ARM
```

这种仅依靠绝对索引 (Index) 移交内存块所有权的设计, 彻底避免了共享内存的锁竞争 (Mutex/Spinlock), 极大降低了系统调度的 CPU 消耗。

4. ALSA 内核流控与抗撕裂机制 (ALSA Flow Control & Anti-Tearing)

这是整个系统能够在高达 100% CPU 负载下依然不爆音、不撕裂的技术核心。



4.1 抗撕裂帧拼装机制 (Offset Assembly)

在高压环境下，ALSA 中断可能被延迟，导致单次 `snd_pcm_readi` 或 `writei` 无法足额获取/写入预定的 960 帧，从而引发 `-EPIPE` (Underrun/Overrun) 或返回截断的残块。系统采用了严密的游标偏移 (Offset) 强制拼装循环：

```

int frames_left = PERIOD_FRAMES; // 坚守 960 帧底线
int offset = 0;
while (frames_left > 0 && g_running) {
    int rc = snd_pcm_readi(handle, tx_base + block_offset + offset, frames_left);
    if (rc < 0) {
        snd_pcm_recover(handle, rc, 0); // 让内核原生修复指针
    } else {
        frames_left -= rc; // 扣减已读帧数
        offset += rc * BYTES_PER_FRAME; // 游标后移，下次继续拼装尾部
    }
}

```

该机制保证了：宁可在 ARM 端死循环等待，也绝不将任何一帧发生移位的残缺数据送入 DSP。

4.2 冷启动预充水机制 (Pre-Charging start_threshold)

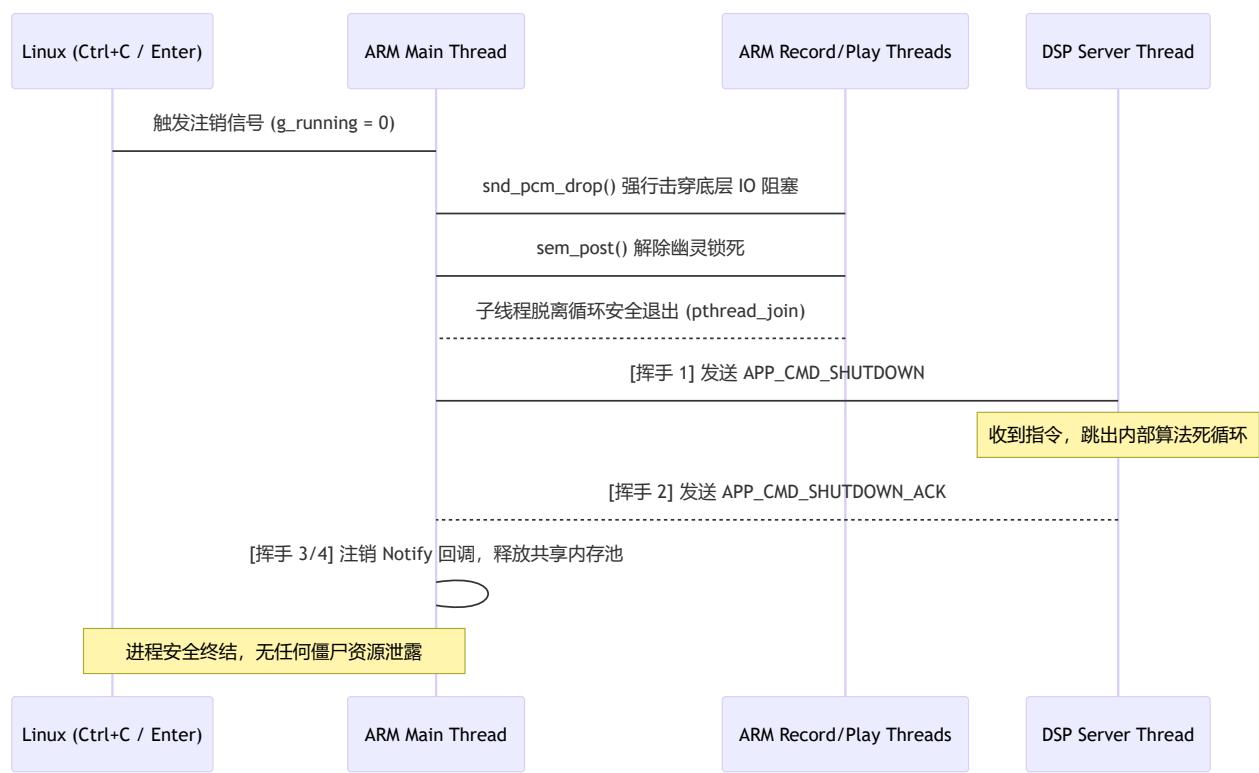
系统上电初期是调度最不稳定的时刻，为了避免播放线程刚喂入一块数据，底层 DMA 就立刻将其抽空引发“缺水爆音”，我们在 `setup_alsa` 中注入了核心防线：

```
// 播放端蓄水池策略：
snd_pcm_sw_params_set_start_threshold(handle, swparams, frames * 3);
snd_pcm_sw_params_set_avail_min(handle, swparams, frames);
```

深度解析： `start_threshold` 强制要求内核引擎在底层硬件 DMA 环形缓冲中**至少积攒够 3 个周期 (60ms)** 的数据后，才允许真正驱动 DAC 发声。这种“先蓄水后开闸”的做法，彻底平滑了启动阶段的调度抖动。

5. 优雅注销与僵尸进程防御 (Graceful Teardown)

嵌入式系统对资源泄漏极度敏感。如果进程异常退出导致 DSP 侧未能释放，或 ALSA 句柄未关闭，下一次启动必将面临硬件死锁。



防御性析构流程 (App_delete) :

- 击穿内核阻塞：** 调用 `snd_pcm_drop(handle)`，将卡在 Linux 内核态苦等 DMA 数据的音频线程瞬间唤醒。

2. **信号量自解**: 调用 `sem_post` 解开 `empty_in` 与 `full_out` 上的幽灵锁死, 让子线程安全退出 `while(g_running)` 循环。
3. **跨核挥手 (SHUTDOWN)**: 向 DSP 发出 `APP_CMD_SHUTDOWN`。DSP 收到后跳出其内部的死循环, 执行清理, 并向 ARM 传回 `APP_CMD_SHUTDOWN_ACK` 确认。
4. **资源核销**: 收到 ACK 后, ARM 主线程才正式注销 IPC 回调, 调用 `Memory_free` 归还共享内存给操作系统。